PG-Match: A DAA-Driven Engine for PG Discovery

Course	BCSE204L — Design and Analysis of Algorithms
Submitted by	Anubhab Baruah (24BPS1152)
Source code	github.com/yago955/daa-assignment

Abstract

PG-Match is a search-and-allocation engine for paying-guest (PG) accommodation, built entirely on six classical DAA algorithms implemented from scratch in Python: KMP string matching, Dijkstra, Floyd–Warshall, 0/1 Knapsack, heap-based top-k selection and Edmonds–Karp maximum flow. On a sample city of 8 localities and 8 PGs, the engine filters listings by amenity, computes shortest commutes, ranks PGs, picks the best add-on bundle within a budget, and allots 20 students to PGs under bed limits.

Two results show why algorithm choice matters. The knapsack DP finds an add-on bundle worth 40 utility points, where a greedy ratio heuristic stops at 36. Max-flow places 10 of the 12 students who have any eligible PG, against 9 for first-come greedy — and max-flow is provably optimal. Benchmarks confirm each algorithm's growth rate: KMP is 157–170× faster than naive matching on adversarial text, and heap top-k is 5–17× faster than a full sort.

1. Problem Statement

Students and young professionals moving to a new Indian city spend days finding a paying-guest (PG) or co-living room. Listings are scattered, descriptions are unstructured, and no platform answers the questions people actually ask: *How long is my commute? Which PG fits my budget with the amenities I care about? Will there even be a bed left?*

PG-Match turns each of those questions into a classical algorithmic problem:

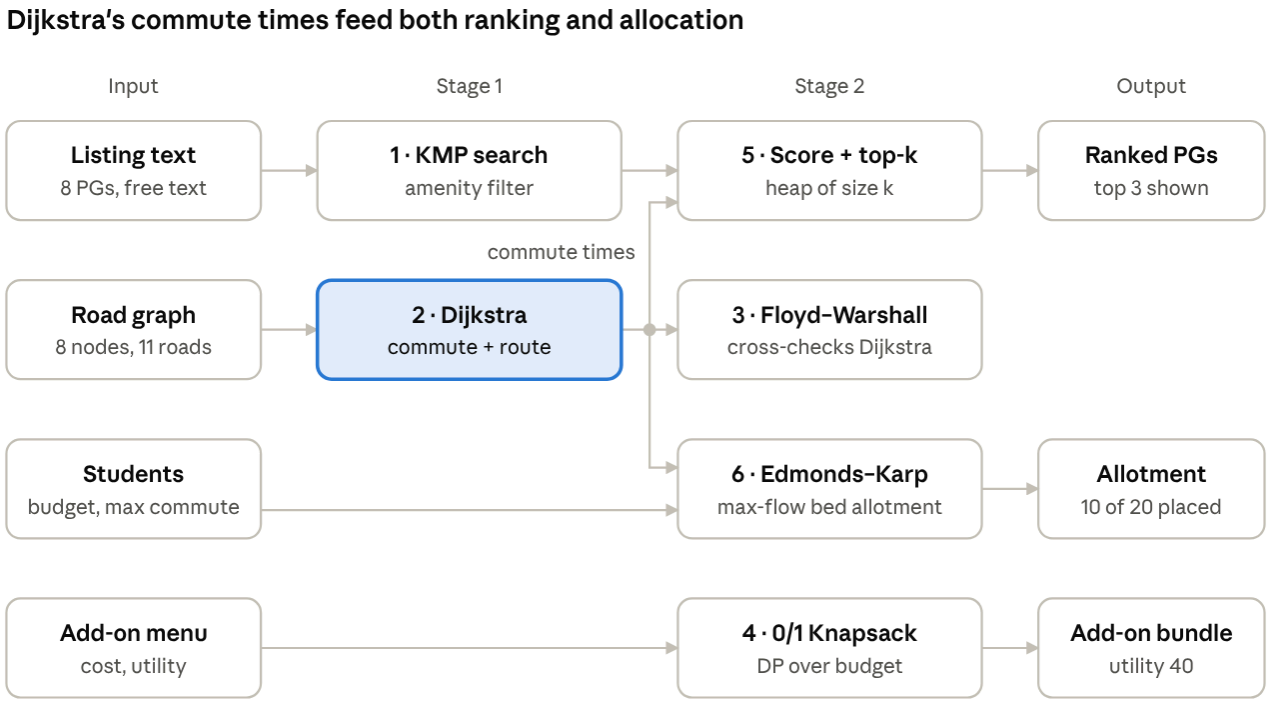
User question	Algorithmic problem	Algorithm used
Which listings mention "AC" or "near metro"?	Exact pattern matching in text	KMP string matching
How far is each PG from my college?	Single-source shortest paths	Dijkstra (binary heap)

User question	Algorithmic problem	Algorithm used
What are commute times between all landmarks?	All-pairs shortest paths	Floyd–Warshall
Which add-on amenities should I pay for within budget?	Constrained optimisation	0/1 Knapsack (dynamic programming)
Which PGs should I see first?	Selection of best k items	Heap-based top-k
How do we allot N students to PGs with limited beds?	Bipartite assignment with capacities	Max-flow (Edmonds–Karp)

The goal is a working engine where every feature is backed by a DAA algorithm with a proven complexity bound, verified empirically on generated data.

2. System Architecture

PG-Match is a two-stage pipeline: stage 1 cleans the inputs (filter text, compute distances), stage 2 makes the decisions (rank, allot, choose add-ons).



PG-Match pipeline · 4 inputs, 6 algorithms, 3 outputs

Dijkstra is the hub: its commute times feed the ranking score and decide which student–PG edges exist in the flow network, while Floyd–Warshall independently verifies them.

3. Algorithms Used

#	Algorithm	Paradigm	Time	Space	Role in PG-Match
1	KMP string matching	String matching	O(n + m)	O(m)	Find amenity keywords in listing text
2	Dijkstra (binary heap)	Greedy	O((V + E) log V)	O(V)	Commute time from campus to every locality
3	Floyd–Warshall	Dynamic programming	$O(V^3)$	$O(V^2)$	All-pairs commute table for landmarks
4	0/1 Knapsack	Dynamic programming	O(n·W)	O(n·W)	Best add-on amenities within a budget
5	Heap top-k	Heaps / selection	O(N log k)	O(k)	Show the k best PGs out of N
6	Edmonds–Karp	Network flow (BFS)	$O(V \cdot E^2)$	O(V + E)	Allot students to PGs with limited beds

3.1 KMP string matching

KMP precomputes a prefix function π, where π[i] is the length of the longest proper prefix of the pattern that is also a suffix of pattern[0..i]. On a mismatch the matcher falls back to π[q−1] instead of restarting, so no text character is compared more than twice. For "abab", π = [0, 0, 1, 2].

Raw matching alone is not enough for real listings: searching "ac" also hits "sp**ac**ious" and "Non AC". PG-Match wraps KMP with a word-boundary check and a negation check ("non" just before the match), which cut 9 raw hits down to the 4 PGs that actually offer AC.

3.2 Dijkstra's algorithm

The city is a weighted undirected graph: localities are vertices, roads are edges weighted by travel time in minutes. Dijkstra grows a set of settled vertices in order of distance, relaxing each outgoing edge:

```
if d[u] + w(u, v) < d[v]:
    d[v] ← d[u] + w(u, v)
    π[v] ← u
```

A binary heap with lazy deletion (stale entries are skipped when popped) gives O((V + E) log V). The parent array π rebuilds the actual route, so the user sees *Campus → Gandhi Nagar → Bus Stand → Old Town*, not just "22 min". Greedy choice is safe because all weights are non-negative.

3.3 Floyd–Warshall

For a table of commute times between every pair of landmarks, Floyd–Warshall allows intermediate vertices 1..k in turn:

```
d(k)[i][j] = min( d(k-1)[i][j],  d(k-1)[i][k] + d(k-1)[k][j] )
```

It runs in Θ(V³) regardless of edge count. On the sample city its "Campus" row matched Dijkstra exactly, which serves as a correctness cross-check between the two algorithms.

3.4 0/1 Knapsack

Each add-on (AC upgrade, gym, laundry, wifi…) has a monthly cost and a utility score. Picking the most valuable set within a budget W is 0/1 Knapsack:

```
K[i][w] = max( K[i−1][w],  K[i−1][w − cᵢ] + uᵢ )    if cᵢ ≤ w
K[i][w] = K[i−1][w]                                 otherwise
```

The table is filled bottom-up in O(n·W), then walked backwards to recover which items were chosen. Costs are in whole rupees, so W is the budget itself; the running time is pseudo-polynomial.

3.5 Heap-based top-k

To show the best k of N scored PGs, a min-heap of size k is kept: each new PG replaces the heap's minimum only if it scores higher. Each step costs O(log k), so the whole scan is O(N log k), against O(N log N) for sorting everything.

3.6 Edmonds–Karp maximum flow

Allocation is modelled as a flow network: source → each student (capacity 1) → each PG the student can afford and reach within their commute limit (capacity 1) → sink (capacity = free beds). The maximum flow equals the maximum number of students that can be housed. Edmonds–Karp finds augmenting paths by BFS in the residual graph, giving O(V·E²). By the max-flow min-cut theorem the result is optimal, which no greedy rule guarantees.

4. Implementation

The engine is a single Python 3 file, `pg_match.py` (about 490 lines). Every algorithm is written from scratch; only `heapq` (a binary heap) and `collections.deque` (a BFS queue) come from the standard library. No external packages are needed.

- **Graph:** adjacency list `adj[u] = [(v, minutes), …]` for Dijkstra; edge list and V×V matrix for Floyd–Warshall.
- **Listings:** each PG is a tuple of name, locality, rent (₹/month), free beds, rating and free-text description.
- **Ranking score (out of 100):** 40 × rating/5 + 35 × affordability (rent scaled between cheapest and costliest) + 25 × (1 − commute/longest commute). Commute comes from Dijkstra.
- **Flow network:** residual capacities stored as a dict per vertex, so each forward edge gets a reverse edge automatically.
- **Baselines for comparison:** naive O(n·m) matching, greedy-by-ratio knapsack, full sort for top-k, first-come greedy allocation, and V runs of Dijkstra versus one Floyd–Warshall.

The sample city — 8 Vellore localities, 11 roads, 8 PGs, 8 add-ons and 20 students — is **synthetic**: travel times, rents and ratings are illustrative, chosen to exercise each algorithm. Student budgets and commute limits are drawn with a fixed random seed (7), so every run is reproducible.

To run:

```
python pg_match.py           # demo on the sample city
python pg_match.py --bench   # demo + timing benchmarks (≈ 1 min)
```

5. Results on the Sample City

All six stages ran correctly on the sample city, and both DP and max-flow beat their greedy alternatives.

5.1 Keyword search (KMP)

6 of 8 listings mention "wifi". A raw KMP search for "ac" returns 9 hits; after the boundary and negation checks, 4 PGs genuinely offer AC: Sri Sai Residency, Nest Co-living, Lotus Ladies PG and Urban Den. "Comfort Stay PG" ("Non AC") is correctly excluded.

5.2 Commute times (Dijkstra, verified by Floyd–Warshall)

Locality	Commute from campus (min)	Shortest route
Gandhi Nagar	6	Campus → Gandhi Nagar
Katpadi Jn	9	Campus → Katpadi Jn
Bus Stand	18	Campus → Gandhi Nagar → Bus Stand

Locality	Commute from campus (min)	Shortest route
Old Town	22	… → Bus Stand → Old Town
Sathuvachari	28	… → Bus Stand → Sathuvachari
Thorapadi	31	… → Bus Stand → Old Town → Thorapadi
Bagayam	33	… → Bus Stand → Old Town → Bagayam

Bagayam is reached via Old Town (33 min), not via Sathuvachari (36 min) — Dijkstra found the cheaper route. The Floyd–Warshall row for Campus matched this table exactly.

5.3 Top-3 PGs (heap top-k)

Rank	PG	Score /100	Rent (₹)	Commute (min)	Rating
1	Comfort Stay PG	75.98	6,000	9	3.9
2	Sri Sai Residency	70.95	7,500	6	4.3
3	Royal Boys Hostel	67.36	5,800	18	3.5

5.4 Add-on bundle (0/1 Knapsack, budget ₹3,500)

Method	Utility	Cost (₹)	Items
DP (optimal)	**40**	3,450	Gym, housekeeping, laundry, wifi, dinner plan, power backup
Greedy by utility/cost	36	2,750	Gym, housekeeping, laundry, wifi, parking, power backup

Greedy grabs cheap parking early and leaves ₹750 unused; DP swaps it for the dinner plan and gains 4 points. This is the classic case where 0/1 Knapsack lacks the greedy-choice property.

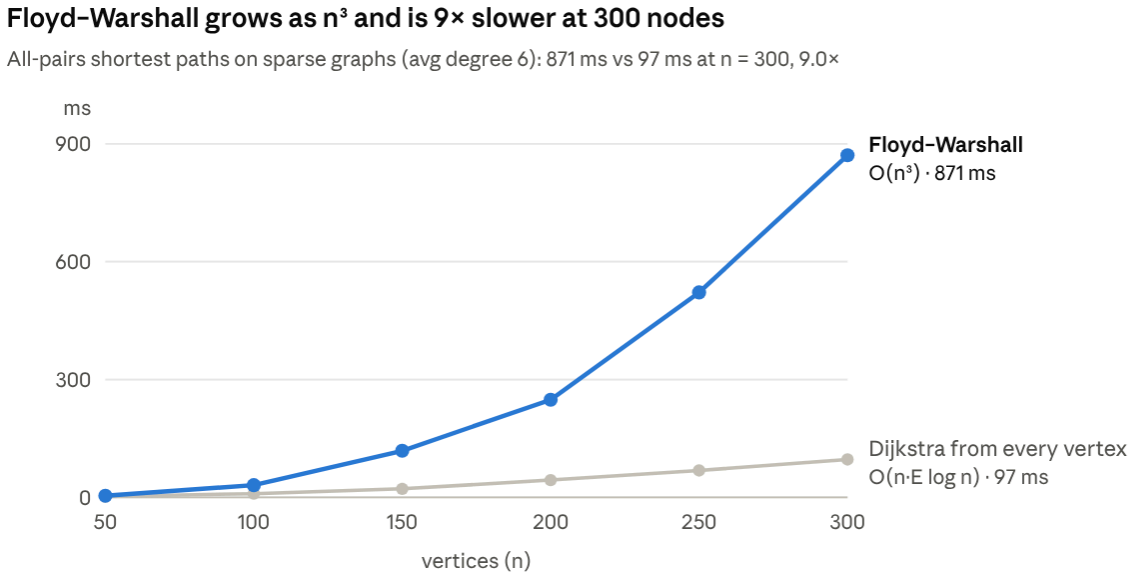
5.5 Student allocation (Edmonds–Karp)

Method	Students placed (of 20)
Edmonds–Karp max flow	**10**
First-come greedy (best-rated eligible PG)	9

There are 25 free beds, but 8 students have no eligible PG at all (budget too low for any PG within their commute limit). Of the 12 who do, max-flow places 10, which is the provable maximum; 2 lose out because their only options are already full. Greedy places 9 because an early student takes a bed that a later student needed more.

6. Empirical Complexity Analysis

Every algorithm's measured growth rate matches its theoretical bound: fitting a straight line to log(time) against log(input size) gives the exponent, and Floyd–Warshall's comes out at 2.99 against a predicted 3.



bench_results.json · random sparse graphs, n = 50 to 300, best of 2–5 runs

For sparse city graphs, running Dijkstra from every vertex beats Floyd–Warshall, and the gap widens with n. Floyd–Warshall is still the right tool for a small, dense landmark table, where its simple triple loop and lack of a heap make it competitive.

Algorithm	Input varied	Range	Expected exponent	Measured exponent
Floyd–Warshall	n	50 → 300	3	2.99
Dijkstra from every vertex	n (E ≈ 3n)	50 → 300	≈ 2 (n·E log n)	2.10
Dijkstra, single source	E	60k → 960k	≈ 1 (E log V)	1.14
KMP search	text length n	5k → 40k	1	1.00

Algorithm	Input varied	Range	Expected exponent	Measured exponent
0/1 Knapsack	n·W	100k → 1.6M	1	1.07
Heap top-k (k = 10)	N	100k → 800k	1 (N log k)	0.82
Full sort (baseline)	N	100k → 800k	≈ 1 (N log N)	1.37
Edmonds–Karp	E	1k → 8k	⩽ 3 ($V \cdot E^2$ bound)	2.11

Key observations:

- **KMP vs naive:** on the adversarial text "aaa…a" with pattern "a…ab" (m = 500), naive matching takes 1,082 ms at n = 40,000 while KMP takes 6.4 ms — about 170× faster, because naive re-scans up to m characters at every shift.
- **Top-k vs sort:** at N = 800,000, the size-10 heap takes 46 ms against 786 ms for a full sort (17×). The full sort's exponent sits above 1 partly because Python's tuple comparison and memory allocation grow with N.
- **Knapsack is pseudo-polynomial:** doubling the number of items (n = 100 → 200 at W = 4,000) took time from 33 to 64 ms, and doubling the budget (W = 4,000 → 8,000 at n = 200) took it from 64 to 144 ms. Cost depends on the numeric value of the budget, not just the number of items.
- **Edmonds–Karp beats its worst-case bound:** with unit capacities on student edges, the number of augmenting paths is at most the flow f, so the real cost is $O(f \cdot E)$. Since f grows with E here, the observed exponent is about 2, well under the $O(V \cdot E^2)$ bound.

All timings are from Python 3.13 on a cloud Linux machine; absolute times will differ on another computer, but exponents should not.

7. Conclusion and Future Work

PG-Match shows that a realistic product — PG search for students — decomposes cleanly into textbook DAA problems, and that the textbook choice wins in practice. DP beat greedy on knapsack (40 vs 36), max-flow beat greedy on allocation (10 vs 9 placed, and provably optimal), and KMP, Dijkstra and heap top-k each scaled at the rate their analysis predicts.

Future work:

- **Min-cost max flow** to place the most students *and* minimise total commute or rent, not just count placements.
- **Gale–Shapley stable matching** when PGs also rank students, so no student–PG pair would rather swap.
- **Aho–Corasick** to search many amenity keywords in one pass instead of one KMP run per keyword.
- **A* search** on real road data (e.g. OpenStreetMap) with straight-line distance as the heuristic.
- **Branch and bound** for roommate grouping under compatibility constraints.

8. References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009 — Ch. 15 (DP), 24 (Dijkstra), 25 (Floyd–Warshall), 26 (Maximum flow), 32 (KMP).
2. D. E. Knuth, J. H. Morris, V. R. Pratt, "Fast pattern matching in strings," *SIAM J. Computing*, 6(2), 1977.
3. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, 1, 1959.
4. J. Edmonds, R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," *Journal of the ACM*, 19(2), 1972.
5. R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, 5(6), 1962.
6. J. Kleinberg, É. Tardos, *Algorithm Design*, Pearson, 2006 — Ch. 6 (DP), 7 (Network flow).

Appendix: source code `pg_match.py`, `sample_output.txt` and `bench_results.json` are attached with this submission.